Team # 27

M2: Mid-submission — Accessible Shopping Layer

Megathon 2026 • Bhashini track

1. Current Approach and Changes since M1

The architecture from M1 is unchanged: a browser extension reads the live shop page into `ProductJSON` and a page map of actions; a FastAPI backend uses AnuvaadHub (ASR, MT, TTS, OCR) and an LLM to return a grounded `AccessibleOutput` plus page actions; the extension acts in the user's own tab, and consequential actions return only as confirmations. The changes below come from testing against the real APIs and real pages.

Track A — Speech and language. One gateway (`bhashini.py`) serves all 21 approved models (en/hi/te; ml/ur ready) with a per-model rate limiter (50 RPM sandbox), calm error mapping, an MT/TTS cache and a background warm-up of every model at start-up. *Forced by the real API:* ASR accepts only WAV, so the extension records 16 kHz mono WAV instead of Opus. ASR returns no confidence or alternatives, so our own mic level tells silence from unclear speech and the LLM's intent confidence drives the 0.8/0.5 rules. TTS returns an S3 link, and MT (≤50 words) and TTS (≤30 words) are chunked automatically. Masking (rule 7) uses `ZQX0`-style placeholders, chosen by experiment; if a placeholder is lost the answer stays in English instead of dropping a price or name. *New:* numbers and prices are spoken as words in the listener's language; a spoken-number normaliser (lakh/crore, dedh/dhai/saadhe, ranges such as "15 se 20 hazaar" → 15000–20000); `translate_many` packs an answer's lines into few MT calls. M1 planned server-side streaming. Instead `/turn` answers in 1–2 s without streaming, and the player speaks sentence by sentence while it prefetches the next sentence's audio. Extension: energy-VAD mic (1.0 s / 2.5 s speech-mode silence timeout, 300 ms pre-roll, muted in secure zones), the audio player with every M1 control, barge-in (except while the device voice plays, because echo cancellation may not remove it), lite mode on the device voice (never an English voice for Hindi/Telugu), and UI strings in en/hi/te with 13 pre-recorded phrases.

Track B — Page understanding and actions. The three-tier extractor (JSON-LD/meta → Amazon.in/Flipkart/mock-store adapters → LLM fallback on redacted visible text) records the source of each field and lists unknown fields in `missing`. *Changes:* the LLM fallback never supplies the price (in testing it took a bank-offer price), and it only fills fields the page lacks. Price, stock, delivery and seller come only from the page. Order numbers never leave the device: the backend sees `ORDER_n`, and these placeholders are now also applied to button labels and LLM text after testing found leaks through "Order MS-1001". Order buttons get ids such as `t-cancel-2`, so a step cannot press another order's button. Address PIN-code fields no longer trigger the secure zone. The page map is capped at 200 controls, with search and cart kept first. A mock store (MockMart: search, product, cart, checkout, orders, order detail) is the only place full checkout, cancel and return run.

Track C — Backend brain. Models: Groq `gpt-oss-120b`/`gpt-oss-20b` (strict JSON, tool calling) for summaries, intent and extraction; Gemini Flash-Lite for photo descriptions. One LLM call per turn picks one of 10 tools. Summaries are LLM-written but verified: every fact must cite a field, code checks each one (numbers, wording, pro/con sentiment, ratings vs. reviews), and a deterministic template is used if the check fails. Comparison ranks products by the user's *stated* preferences, which are remembered per session. Weights never come from the disability profile. Track, cancel, return and place order are step machines. Place order reads back the total and delivery PIN, which needed a new `CheckoutSummary` schema. New endpoint `/extract/fallback` for Track B. Server streaming was dropped, as explained under Track A.

Track D — Accessibility UX. Built as planned: an on-device overlay with one live region and renderers, a rule-based composer whose input ladder always ends at one switch, two-level switch scanning, phrase board, local matcher, on-device inference (suggest → 10 s countdown → undo), simplified view, action menu, colour-swatch names and a step-by-step checkout wizard. *Changes:* the matcher now uses word-level edit distance instead of Fuse.js (see experiment). The two-signal rule is built into scoring, and keyboard-only use triggers one clarifying question. Simple-mode speech speed is now a starting default (0.75×) that the user can change. The first-run setup is a separate page, not a pop-up on shop pages. Settings show which modes were inferred and which the user chose. Alt+Shift+A opens the assistant. *Integration since M1:* the overlay now uses Track A's player, mic and barge-in; secure-zone muting and Track B's extraction fallback are wired in, all on `main`.

2. Experiments and Results

All language and LLM experiments ran live (AnuvaadHub, Groq, Gemini); page experiments used 10 saved real pages (5 product, 4 search, 1 cart; Amazon.in and Flipkart, logged out, personal data removed).

Experiment	Result
Track A — language	
Round trip MT→TTS→ASR→MT	Exact in en/hi/te. The listed Telugu ASR model never answered (60 s), so we tried 3 others: 1 exact in 0.9 s (adopted), 1 with a spelling slip, 1 hung.
Latency	MT 0.1 s warm / 6–10 s cold; ASR 0.7–6 s; TTS 3–19 s per sentence (Telugu slowest); OCR ∼1 s. Hence the warm-up, prefetching and pre-recorded phrases.
Masking placeholders (7 styles × 3 directions)	⟦0⟧ made MT invent numbers; `<0>`, `{0}`, `__0__` failed in some direction; `ZQX0` survived all three. Unmasked MT: "Bluetooth 5.3"→"5", "6.8 inch"→"6.80", brands transliterated; masked: every name, unit and price exact.
How TTS reads numbers	Hindi/Telugu voices read digits in English and dropped the rupee sign. After conversion to words, ASR heard "satrah hazaar nau sau ninyaanve rupaye" (and the Telugu equivalent).
Packing lines into one MT call	Newlines merged lines; "‖" became a double danda; a marker word moved mid-sentence (silent corruption). A numbered list kept every item: a 14-line answer in 2 calls (0.4 s hi, 1.1 s te) instead of 14.
OCR; burst; UI strings	OCR misreads units ("5000 mAh"→"500 mAb"), so OCR is never a source of facts. ∼20 simultaneous calls → HTTP 500, so warm-up runs 2 at a time (20/21 models in 26 s). 14 of 44 machine-translated hi+te UI strings needed fixes; a native Telugu speaker reviewed the result and asked for no further changes.
Track B — extraction	
Page type, schema	Page type correct on 10/10 pages; extracted data passed the JSON Schema check on 10/10.

Product pages	Amazon (3): all 12 top-level fields filled, nothing missing; 33–58 specs with parsed units, 3–7 variants, 7 images, 8–13 reviews. Flipkart (2): 33 and 50 specs, up to 4 variants. 4 fields absent from the page are correctly listed in `missing`, not guessed.
Search pages; speed	Result cards read: Amazon earphones 22, cookers 56; Flipkart earphones 40, mobiles 24. Extraction 40–240 ms per page (simulated DOM, not a real browser). Fixed: hidden pop-up heading taken as title, footer links as Flipkart variants, search box cut by the 200 cap, order numbers leaking in labels.
Mock-store flows	Search→product→add to cart (verified by cart count); "Place order" refused until confirmed; typing into card fields refused; cancel/return each ask for a reason.
Track C — brain	
LLM summaries on 5 real pages	End to end over HTTP in ∼2–4 s. Found and fixed: ratings reported as reviews, misleading review counts, raw numbers read aloud, mixed review topics lost, an advert taken as a fact. Final: **0 unsupported claims**.
Hallucination checks	Invented specs ("6000 mAh"), wrong-sentiment pros and unsupported opinions are dropped by the verifier.
`/turn` intent and safety (14 utterances)	English, Hinglish, unclear speech and noise all handled correctly in 1–1.7 s per turn: "isse cart mein daal do" → add-to-cart confirmation; "buy it now" → refused; confidence 0.41 → "Did you mean Open the cart, or Add to cart?"; noise → phrase chips.
Photos; extraction fallback	Gemini descriptions in 3.6–6.9 s; banner text ("IP67 Rating", "50MP") correctly dropped as a claim. Fallback: rating 5/5, seller 5/5, an advert now rejected, price never mis-extracted.
Order workflows; live Hindi	Track, return-with-reason, cancel and place order all correct on the mock store, risky steps only through high-risk confirmations. Hindi `/turn` with Track A's real MT in 1.1 s, brands and prices exact.
Track D — accessibility UX	
Local matcher: Fuse.js vs. word-level	Fuse.js scored "no" vs. product "Novix A15" at 0.90 (near the real "no" command), "nex" 1.00 for "next", "add to card" only 0.75. Word-level: "add to card"→Add to Cart 0.88 (acts); garbled words ("paws" 0.48, "slowa" 0.64, "nex" 0.66) go to the server instead of acting.
Browser voices	Chrome 149/Linux: 19 voices, all Google *online* (en, hi; no Telugu; text goes to Google, which goes on our Responsible AI slide); 0 with `-enable-speech-dispatcher`. Firefox uses on-device voices. A language is never read with the wrong voice.
axe-core; button audit	0 violations in 7 overlay states (colour contrast still needs a real browser). Every button in 6 states clicked automatically: 2 bugs fixed.
Composer; inference	All 16 condition combinations meet the invariants (switch always last, values in range, adding a condition never loosens pacing), including the older blind user with a tremor. Single-signal evidence never triggers a suggestion (unit tests).

3. Progress on Testing

- **Automated (`make check` on every PR, all green on `main`):** 521 backend tests (3 live-API tests skipped without keys) and over 430 extension tests, plus every fixture validated against the JSON Schemas, lint and type regeneration. This includes table-driven masking (58 cases), the number normaliser and spoken numbers (∼140 cases), extraction on every saved page, mock-store flows, the composer, matcher, axe-core and button audits, and the player, mic, VAD and barge-in run on synthetic audio. Real Groq/Gemini/AnuvaadHub responses are recorded as fixtures; a repeatable live smoke test (`smoke_language_api.py`) covers the language APIs.
- **M1 safety tests, done:** no order without explicit confirmation; buy/pay buttons never pressed by the LLM; password/card/CVV/OTP/UP PIN fields and payment frames trigger the secure zone (mic muted, nothing extracted, turn stopped); Luhn-checked cards, Verhoeff-checked Aadhaar, PAN, OTPs, order numbers, emails and phones redacted before any LLM or MT call; no price from the LLM fallback.
- **Bugs found by tests and fixed (examples):** 6 wrong normaliser outputs ("three hundred and ninety nine"→"390 9"); Telugu number grammar; Hindi/Telugu vowel signs stripped in the matcher; sentence splitting broken by numbered items; a Chrome flag silencing all voices; and the extraction and summary bugs listed above.
- **Manual:** an overlay walkthrough (opening, focus, Escape, announcements) led to "Back to menu", live speed control, honest demo-mode messages and a fully localised UI.
- **Not yet done:** runs in real Chrome and Firefox for Android (mic permission, autoplay, echo cancellation); NVDA/ChromeVox/TalkBack passes; ASR on real human recordings (so far only TTS-generated speech); per-field extraction accuracy against hand labels (M1 plan); inference false-suggestion rate (tooling ready, no sessions recorded yet); the full voice→answer loop latency.

4. Plan for the Remaining Time

1. **End-to-end integration (all):** extraction at load, `/turn` and `/product/prepare`, actions and confirmations in one loop; then persona runs for every required combination (visual+motor, visual+speech, speech+motor, ageing+any) with full-loop latency measured against the 4 s budget.
2. **Real devices:** mock store and real sites in Chrome and Firefox for Android; real extraction time; autoplay, barge-in echo and device voices (Telugu often missing on desktop); screen-reader passes and colour contrast.
3. **Checkout and orders (B, C):** extract `CheckoutSummary` on the mock store, then Amazon/Flipkart (read-only on real sites); save logged-in cart/order pages; learned adapters from the LLM fallback.
4. **Evaluation for the final slides:** per-field extraction accuracy on hand-labelled pages; real en/hi/te command clips in `fixtures/audio/` as a round-trip test; inference sessions from each teammate, weights tuned to zero false suggestions, rate and time-to-correct-suggestion reported; TORGO command accuracy, plain ASR vs. our full stack (stretch).
5. **Demo readiness:** demo script on the mock store (search → compare → cart → place order, plus a Hindi run); renew the AnuvaadHub token (expires 13 Oct, 10:00 IST); keep warm-up, pre-recorded phrases and the device voice as fallbacks. Stretch: Malayalam and Urdu, personal voice enrollment.